Quantum-Assisted Knowledge Distillation for Lightweight Waste Classification: A Comparative Study

Author names
Affiliation

Abstract*—Knowledge distillation (KD) suits camera-assisted waste sorting on low-power hardware: a large teacher guides a compact student, and only the student is deployed. A recent proposal adds a quantum kernel to the distillation loss. Quantum computation is then needed only during training, and the deployed network stays classical. Whether such a kernel helps beyond kernel alignment itself requires matched classical controls. We test this on TrashNet, with RPS-25 as a low-data confirmation. The setup uses a ResNet-50 teacher, three students (LeNet-5, MobileNetV2-0.35, MobileNetV3-Small) and one shared training pipeline. We compare hard-label training, logit KD, relational KD, a classical RBF-kernel twin and a projected quantum kernel (PQK). Each cell has three seeds (87 grid runs; 152 student runs including controls, screens and capacity probes). PQK is not distinguishable from logit KD. It is ahead on the mean in 3 of 6 student–dataset pairs and behind on both TrashNet MobileNets, and no paired difference is significant (smallest uncorrected $p = 0.22$). At its pre-specified kernel scale, PQK beats its RBF twin in only 2 of 6 pairs. Its seed-to-seed variance tends to be lower than that of the RBF twin and, less consistently, of hard-label training. This is not significant at three seeds and does not hold on every pair. All 66 exported MobileNet students convert to fully integer INT8 models of about 0.62 and 1.21 MB. On a Raspberry Pi 4B they run in 2.5 and 3.2 ms per inference (median, four threads). INT8 conversion changes macro-F1 by $+0.003$ on average (std. 0.022 across models), and the sign of per-method effects depends on which INT8 kernels run. The quantum kernel leaves no trace in the deployed model, but our evidence does not show an accuracy benefit over classical alternatives. All results are exploratory.

I. Introduction

Sorting waste at the point of disposal, for example in a camera-assisted bin or collection station that suggests a category for the item presented to it, needs a classifier that runs locally on inexpensive hardware, with a small memory footprint and low latency. The data for such a task are small and imbalanced. TrashNet [1], the dataset we use, has six material classes whose training counts range from 96 to 416 images in our split. Compact convolutional networks such as MobileNets [2, 3] and 8-bit integer inference [4, 5] address the footprint, at some cost in accuracy relative to large networks.

Knowledge distillation (KD) [6] recovers part of that accuracy. The compact student is trained to match a larger teacher's output distribution in addition to the hard labels. Relational and feature-based variants transfer the geometry of the teacher's representation instead of, or in addition to, its outputs [7–9]. Only the student is deployed, so distillation is compatible with quantization, and the two are complementary compression mechanisms [10, 11].

A recent line of work asks whether quantum computation can assist distillation while the deployed model stays classical. Quantum relational knowledge distillation (QRKD) [12] extends relational KD. It maps features into a Hilbert space, treats them as quantum states, and uses quantum-kernel values to describe relations between samples. It reports gains over classical relational KD on CNNs for MNIST and CIFAR-10 and on GPT-2 language tasks, with quantum computation needed only in training. Other work distills classical teachers into quantum students [13, 14] or uses a quantum model as a soft-label teacher for a classical student [15]. Our question is narrower: does the quantum component contribute anything that a classical kernel in the same place would not?

For kernels, that question has no obvious answer. Quantum kernels can concentrate, so that kernel values stop distinguishing inputs [16]. Numerical studies of fidelity kernels on classical data find no advantage [17]. The inductive bias of a quantum kernel determines what it can learn [18], and once the bandwidth is tuned it can resemble a classical kernel [19]. Large-scale benchmarks of fidelity and projected quantum kernels examine which design choices make them work [20]. The projected quantum kernel (PQK) of Huang et al. [21] compares inputs through single-qubit reduced states of an embedding circuit. It is a clean candidate to test: at the small scale used here it can be simulated classically, and a classical kernel acting on the same inputs can replace it one-for-one.

Two practical points shape how such a comparison has to be run. First, effects on small datasets are comparable in size to seed-to-seed noise. Conclusions therefore need several seeds, paired comparisons, and an explicit statement of what was selected on which split. Second, the end point is the deployed INT8 model, and INT8 results depend on the kernels that execute it.

We therefore compare quantum-assisted distillation with classical alternatives under one pipeline. TrashNet (six classes, imbalanced, macro-F1) is the primary dataset. RPS-25, a 25% training subset of Rock–Paper–Scissors [22] (three classes), is a low-data confirmation. A frozen ResNet-50 teacher [23] distills into three students of 0.03 M to 0.93 M parameters (LeNet-5, MobileNetV2-0.35, MobileNetV3-Small). Five methods are compared: hard-label training (`scratch`), logit KD (`kd`), relational KD (`rkd`), classical RBF-kernel alignment (`rbf_control`, the matched twin) and PQK (`pqk`). All methods on a pair share data, optimizer and epoch budget.

Each cell has 3 seeds, and every exported model is a plain INT8 network with no quantum operation. Two further arms isolate where a PQK effect could come from: the same circuit with its CNOT gates removed, and a fitted instead of random teacher projection.

Our findings are as follows.

- The pipeline can detect a distillation gain: `kd` beats `scratch` on TrashNet for both MobileNets (by 0.025 and 0.009 macro-F1).
- `pqk` is not distinguishable from `kd`. It is ahead on the mean in 3 of 6 pairs (4 if one seed trained with a different KD temperature is excluded) and behind on both TrashNet MobileNets (by 0.025 and 0.014). The smallest uncorrected paired p-value is 0.22.
- At its pre-specified kernel scale, `pqk` beats its RBF twin in 2 of 6 pairs (smallest uncorrected $p = 0.13$). With a median-heuristic scale, adopted after these results were seen, it does so in 3 of 4. The one nominally significant difference ($p = 0.01$, RPS-25/MobileNetV3-Small) favours the classical twin.
- QRKD reports consistent gains over classical relational KD. Our setting differs in data, students and loss. There, `pqk` is ahead of `rkd` in 2 of 5 pairs, by at most 0.012 macro-F1, and behind in the rest.
- On macro-F1, `pqk` has lower seed-to-seed variance than `rbf_control` in 4 of 6 pairs and than `scratch` in 4 (3 if one recipe-mismatched run is excluded). On top-1 the counts are 5 and 5. This is not significant at three seeds, and `rkd` is about as stable.
- Deployment does not depend on the training method: all 66 exported MobileNet students are fully integer, per-channel INT8 models, and INT8 conversion changes macro-F1 by +0.003 on average. The size of each method's INT8 effect depends on the runtime kernels.

All comparisons are exploratory. Cells have three seeds, and test splits informed earlier design choices. We therefore select on validation, report paired differences and treat test scores as descriptive.

a) Organization.: Section II reviews related work. Section III describes the objectives, the quantum kernel and its controls, and Section IV the data, training and deployment protocol. Section V reports the results.

b) Contributions.: (i) A matched-control comparison of quantum-assisted and classical distillation for lightweight waste classification, with one pipeline, a classical RBF-kernel twin for the quantum kernel, and paired per-seed reporting. (ii) Two ablations that probe where a quantum-kernel effect could originate: removing the circuit's entanglement, and replacing the random teacher projection by a fitted one. The fitted projection makes the target kernel nearly the label kernel on training batches and did not help. (iii) Deployment evidence for the resulting students: a fully integer INT8 export of all 66 models, on-device latency, memory and temperature on a Raspberry Pi 4B, and a measure of how INT8 results depend on the kernels that run them. (iv) An account of the pitfalls we hit and corrected, including a collapse of the kernel-alignment loss when the teacher-side projection is trainable, and of how our own selection choices limit what the results can support.

II. Related Work

a) Knowledge distillation and relational transfer.: Logit distillation trains a student on a teacher's softened output distribution [6]. FitNets add hints from intermediate representations so that thinner students can follow deeper teachers [7]. Relational knowledge distillation (RKD) transfers the structure among samples rather than single outputs. It aligns pairwise distances and angles of the embeddings [8]. Contrastive representation distillation and instance-relationship graphs pursue the same idea with different objectives [9, 24]. A kernel view connects these methods. A trained network's predictions are determined by the inner products of its last-layer representations, the feature kernel, and that kernel is a natural object to distill [25]. The kernel-alignment loss used here belongs to this family. It matches batch Gram matrices of the teacher and the student after projecting both to eight angles, and its classical twin replaces only the kernel that defines the Gram matrices.

b) Quantum kernels and their limits.: Quantum kernels embed inputs in the Hilbert space of a quantum system and compare them there [26, 27]. The projected quantum kernel of Huang et al. [21] compares reduced states of the embedded system instead of the full state. Whether quantum kernels help on classical data is contested. Kernel values can concentrate exponentially. Entanglement, noise and embedding expressivity are among the identified sources [16]. Fidelity kernels also suffer a related vanishing-similarity problem, which alternative constructions try to mitigate [28]. Numerical studies find no advantage of tuned fidelity kernels on classical data [17]. Analyses of noise and finite data identify regimes where an advantage disappears [29]. Bandwidth-tuned fidelity and projected quantum kernels have been argued to behave classically [19]. The inductive bias of the embedding determines what a quantum kernel can learn [18]. A hyperparameter study examines how quantum-kernel design choices affect performance and the generalization gap across datasets [30]. A large-scale benchmark of fidelity and projected quantum kernels studies which design choices make them effective [20]. Radial basis function networks have been extended to quantum kernel functions [31]. These results motivate our design. The quantum kernel is paired with a classical RBF kernel on the same inputs, and a variant of the quantum kernel receives the same median-heuristic scale as its twin. The comparison can then isolate the kernel rather than its calibration.

c) Quantum-assisted distillation.: Several studies combine distillation with quantum models. Quantum relational knowledge distillation (QRKD) [12] extends RKD with quantum-kernel values computed from features mapped into a Hilbert space. It evaluates CNNs on MNIST and CIFAR-10 and GPT-2 on three language benchmarks, and reports consistent gains over classical RKD with all deployed models classical. In the other direction, classical teachers have been used to improve the training accuracy and stability of quantum students [13].

One study covers 4- and 8-qubit quantum neural networks on MNIST, Fashion-MNIST and CIFAR-10, with an analysis of dimensionality reduction and data encodings [14]. A variational circuit has also served as an offline soft-label teacher for a compact classical student, with reported robustness gains on MNIST [15]. Quantum-inspired latent-space compression has been combined with distillation for resource-constrained deployment [32]. Our setting is closest to QRKD, with quantum computation confined to training. It differs in the data (small imbalanced waste images), the students (0.03–0.93 M parameters), the loss (kernel alignment in addition to a logit term), the controls (a classical RBF twin and two ablation arms), the reporting (three seeds, paired comparisons) and the end point (INT8 models measured on a Raspberry Pi). We do not claim that the results of those studies fail in their own settings.

d) Distillation, quantization and edge deployment.: Distillation and quantization are complementary compression mechanisms [10], and quantization-aware distillation trains the two jointly [11]. Integer-arithmetic-only inference [4] and its practical recipes [5] are the basis of the 8-bit deployment used here. We apply post-training quantization to finished students instead of training with it. Edge-oriented distillation work studies accuracy–efficiency trade-offs for CNNs and vision transformers [33], evaluates distillation and compression on edge-class devices [34], and splits distilled networks between device and edge server [35]. We take the same deployment-oriented view but measure a single compact INT8 model on the device itself, and we report how its predictions depend on the kernels that execute it.

III. Method

Let x be an image with label y, let $z_t(x) \in \mathbb{R}^{2048}$ and $z_s(x) \in \mathbb{R}^{d_s}$ be the penultimate features of the teacher and the student, and let ℓ_t, ℓ_s be their logits. All losses are computed on batches of up to $B = 128$ images; the last batch of an epoch is smaller, and the 33-image RPS-25 validation split is a single batch.

A. Baseline objectives

`scratch` minimizes the cross-entropy $\mathrm{CE}(\ell_s, y)$. `kd` is Hinton distillation [6],

$$L_{\mathrm{KD}} = \alpha\,\mathrm{CE}(\ell_s, y) + \beta\,\tau^2\,\mathrm{KL}(\mathrm{softmax}(\ell_t/\tau)\,\|\,\mathrm{softmax}(\ell_s/\tau))\,, \tag{1}$$

with $\alpha = \beta = 0.5$ and a temperature τ that was screened on some pairs and reused on others (Table IV). `rkd` [8] adds distance-wise and angle-wise relational terms on the penultimate features,

$$L_{\mathrm{RKD}} = \mathrm{CE}(\ell_s, y) + w_d\,\mathrm{SmoothL1}\big(\psi_D(z_s), \psi_D(z_t)\big) + w_a\,\mathrm{SmoothL1}\big(\psi_A(z_s), \psi_A(z_t)\big), \tag{2}$$

where ψ_D collects the pairwise distances of a batch normalized by their mean and ψ_A the cosines of the angles formed by triplets. Both potentials are dimension-free, so a 2048-dimensional teacher and a student of any width can be compared directly.

B. Kernel alignment

The two kernel methods (`pqk` and its twin `rbf_control`) add a kernel-alignment term to the logit objective,

$$L = \alpha\,\mathrm{CE}(\ell_s, y) + \beta\,\tau^2\,\mathrm{KL}(\cdot) + \gamma_e\,L_{\mathrm{kernel}}, \tag{3}$$

$$L_{\mathrm{kernel}} = \big\|\tilde{K}_t - \tilde{K}_s\big\|_F^2, \qquad \tilde{K} = \frac{K}{\mathrm{tr}\,K}, \tag{4}$$

with $\alpha = \beta = 0.5$, a single temperature $\tau = 0.5$ for all kernel methods, and the weight $\gamma_e = \gamma_{\max}\min\big(1, (e{+}1)/10\big)$ ramped linearly over the first ten epochs e, with $\gamma_{\max} = 1$. Validation is always scored at $\gamma_{\max}$ so that the ramp cannot change which checkpoint is selected. K_t and K_s are the $B \times B$ Gram matrices of the teacher and the student, built from eight angles per image,

$$\theta_s = \pi\,\sigma(W_s z_s + b_s), \quad \theta_t = \pi\,\sigma(W_t z_t + b_t), \quad \theta \in [0, \pi]^8. \tag{5}$$

The student-side projection (W_s, b_s) is trained jointly with the student, and gradients reach the student through the kernel. The teacher-side projection (W_t, b_t) is drawn at default initialization at the start of each run, frozen, and evaluated without gradients. Freezing it is necessary. If both projections are trainable, collapsing both to constant maps sets every Gram entry to one and makes the two normalized matrices identical. The loss L_{kernel} then reaches exactly zero without the student learning anything. Our first implementation had this flaw. In the first `pqk` run we inspected (RPS-25, LeNet-5, seed 0) the mean off-diagonal kernel value was 0.995 for the student and 0.986 for the teacher. We log the mean off-diagonal kernel value in every epoch and treat values below 0.05 or above 0.95 as degenerate.

C. The projected quantum kernel

The quantum kernel is the projected quantum kernel of Huang et al. [21]. The circuit (Figure 2) applies $R_Y(\theta_q)$ to every qubit q and then a CNOT from qubit q to $q{+}1$ for $q = 0, \ldots, 6$, and repeats this pair of layers 4 times on 8 qubits starting from $|0\rangle^{\otimes 8}$. The CNOTs form an open chain, with no gate closing it into a ring. For each input the Bloch vector of every qubit is measured, $r(\theta) = (\langle X_q\rangle, \langle Y_q\rangle, \langle Z_q\rangle)_{q=0}^{7}$. Because R_Y and CNOT are real matrices, the state stays real and $\langle Y_q\rangle \equiv 0$, so only 16 of the 24 coordinates are active. The kernel compares inputs through their single-qubit reduced states $\rho_q = \frac{1}{2}(I + r_q \cdot \sigma)$,

$$K(\theta, \theta') = \exp\Big(-\lambda \sum_q \big\|\rho_q(\theta) - \rho_q(\theta')\big\|_F^2\Big) = \exp\Big(-\tfrac{\lambda}{2}\big\|r(\theta) - r(\theta')\big\|^2\Big), \tag{6}$$

where the second form follows from $\mathrm{tr}(\sigma_a\sigma_b) = 2\delta_{ab}$. The readout is restricted to single-qubit expectation values. At eight qubits the circuit is cheap to simulate exactly, so the kernel is classically tractable at this size. We use PennyLane [36] on its state-vector `lightning.qubit` backend with adjoint differentiation.

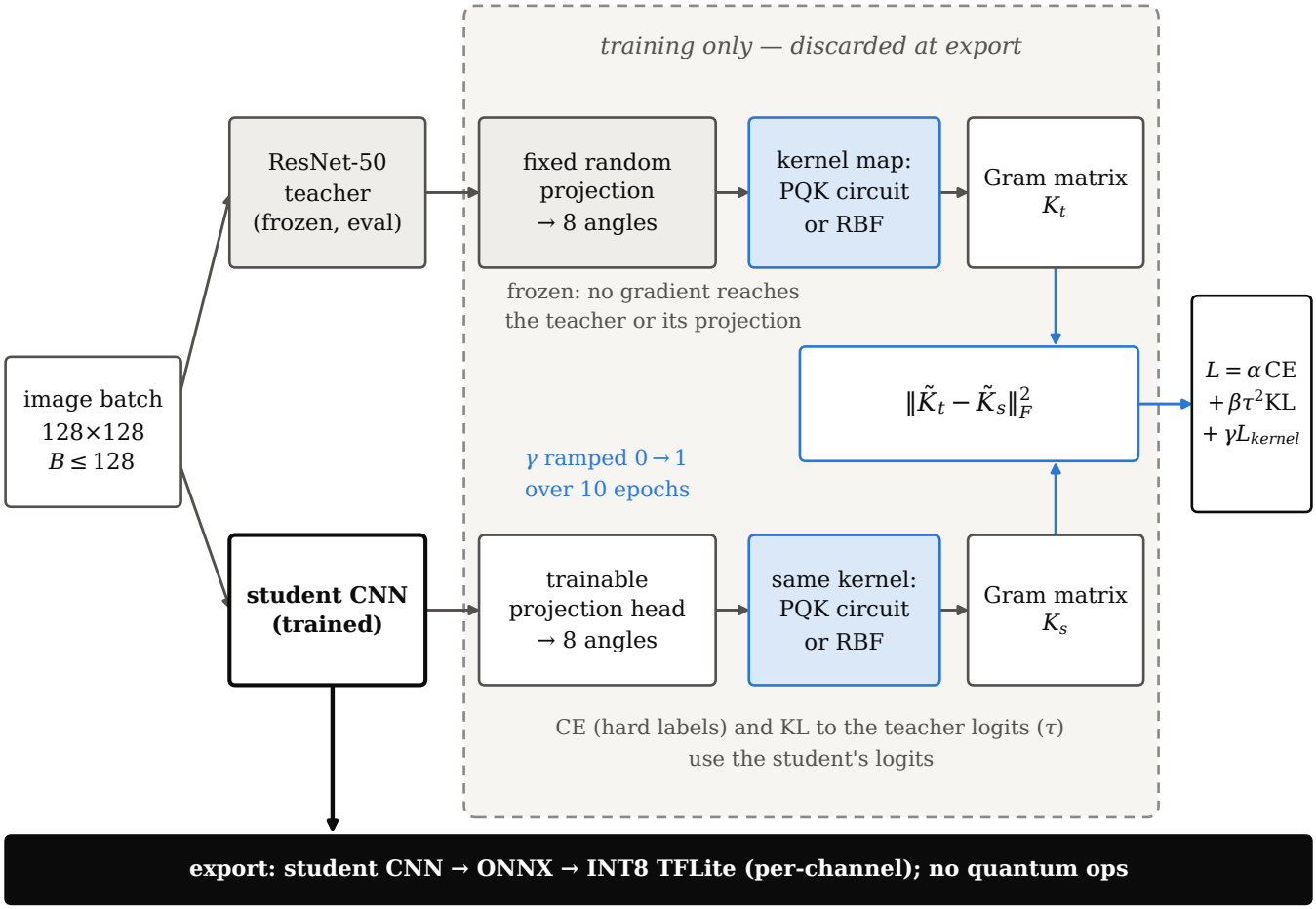


Fig. 1. Training-time pipeline. Penultimate features of the frozen teacher and of the student are projected to eight angles and compared through the Gram matrices of one kernel (the PQK circuit or an RBF kernel). Everything inside the dashed region is discarded at export; only the student CNN is deployed.

The scale λ is handled in two ways. `pqk` uses the pre-specified $\lambda = 1$. The variant `pqk_adaptive`, adopted after the first results (Section IV-D), sets λ in every batch so that $K = \exp\big(-d^2/(2m)\big)$ with $d^2 = \frac{1}{2}\|\Delta r\|^2$ and m the median off-diagonal d^2 of the batch. With this choice the Gram matrix is the median-heuristic RBF kernel on the measured features $r(\theta)$.

D. Matched classical controls

`rbf_control` is the same pipeline with an RBF kernel, $K = \exp\big(-\|\theta - \theta'\|^2/(2m)\big)$, on the same eight angles with the same median heuristic. The quantum branch with adaptive λ and `rbf_control` therefore differ only in the map $\theta \mapsto r(\theta)$ that precedes an identically calibrated RBF kernel. Two further arms isolate where an effect of that map could come from.

- `noent_control` deletes the CNOT gates from the circuit. Each qubit then sees $R_Y(\theta_q)^4 = R_Y(4\theta_q)$, so $r = (\sin 4\theta_q, 0, \cos 4\theta_q)$ exactly (checked against the simulator to 6×10^{-8}), with the same 16 active coordinates and range, no trainable parameters and the same adaptive RBF kernel. It differs from `pqk_adaptive` only by entanglement.
- `pqk_fit` and `rbf_fit` replace the random teacher-side projection by a map fitted once on the training split and then frozen: the $C-1$ shrinkage-LDA directions, completed to eight outputs with the leading principal components of the residual and standardized on the training set.

A third control, random Fourier features drawn from the circuit's own frequency spectrum, was implemented but not run. Table I lists the settings of every kernel arm.

E. What is trained and what is deployed

Only the student CNN is exported. The projection heads, the circuit and the kernels exist only inside the training loop, and a purity check on the exported graph fails if any of them is present (Section IV-E). The deployed model of every method, including `pqk`, is therefore an ordinary convolutional network, and the quantum kernel affects it only through the weights it learned.

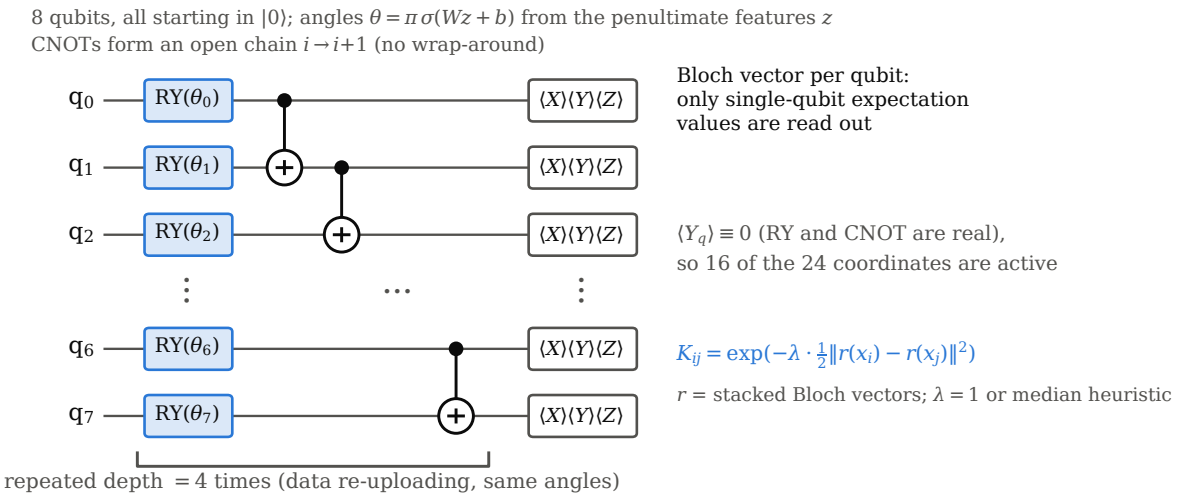


Fig. 2. The fixed feature map behind `pqk` (shown for qubits 0, 1, 2 and 7). Each of the eight angles drives an R_Y rotation, followed by an open chain of CNOT gates; the pair is repeated 4 times with the same angles. Only single-qubit expectation values are read out. Nothing in the circuit is trainable.

TABLE I
KERNEL ARMS. ALL USE EIGHT ANGLES, THE LOSS OF EQ. (3) AND THE SAME FROZEN TEACHER.

Method	Kernel on the 8 angles	Qubits	Depth	λ / bandwidth	Teacher projection
`rbf_control`	RBF	8	–	median heuristic	random (frozen)
`pqk`	projected quantum (RY+CNOT)	8	4	1	random (frozen)
`pqk_adaptive`	projected quantum (RY+CNOT)	8	4	median heuristic	random (frozen)
`noent_control`	classical map $[\sin 4\theta, \cos 4\theta]$	8	4	median heuristic	random (frozen)
`pqk_fit`	projected quantum (RY+CNOT)	8	4	median heuristic	LDA+PCA (fitted, frozen)
`rbf_fit`	RBF	8	–	median heuristic	LDA+PCA (fitted, frozen)

IV. Experimental Setup

A. Data

TrashNet [1] has 2,527 photographs in six classes (cardboard, glass, metal, paper, plastic, trash) and no official split. We split it once, stratified 70/15/15 with seed 0, cache the split, and never re-split. The classes are imbalanced (Table II). Macro-F1 is therefore the primary metric, with accuracy as secondary, and training batches are drawn with a class-balanced sampler instead of class-weighted cross-entropy. **RPS-25** is a 25% subset of the training images of the Rock–Paper–Scissors dataset [22]. We chose it to be deliberately low-data so that distillation has room to help. It keeps the official validation (33 images) and test (372 images) splits, and top-1 accuracy is its primary metric. The two datasets differ in nature as well as size. RPS-25 is a three-class gesture task with clean backgrounds, and TrashNet is a six-class material task with imbalance. We treat RPS-25 as a low-data confirmation, not as a second application. All images are resized to 128×128 and normalized with ImageNet statistics. Training images use random resized crops, RandAugment [37] and horizontal flips, with the strengths in Table III. These are identical for every method.

B. Teacher and students

The teacher is an ImageNet-pretrained ResNet-50 [23], fine-tuned once per dataset and then frozen in evaluation mode with gradients disabled. Fine-tuning uses AdamW [38] with learning rate 10^{-4}, weight decay 10^{-4}, a cosine schedule, batch 128, at most 20 epochs and patience 5, and selects by validation macro-F1. Its test macro-F1 is 0.876 on TrashNet and 0.907 on RPS-25. The students are trained from random initialization. They are MobileNetV2 with width multiplier 0.35 (0.40 M parameters), MobileNetV3-Small (0.93 M) and a LeNet-5 [39] variant with global average pooling before the classifier (0.03 M). MobileNetV2-0.35 and MobileNetV3-Small are the edge-relevant students and the only ones exported. LeNet-5 serves as a capacity floor.

C. Training protocol

All methods on a pair share the data, the optimizer (AdamW, weight decay 10^{-4}, cosine learning-rate schedule), the batch size (128), the epoch budget and the early-stopping patience



Fig. 3. Training images from the two datasets at the 128×128 input resolution, two per class, and the number of training images per class.

TABLE II
DATASETS AND SPLITS.

Dataset	Classes	Train	Val	Test	Smallest / largest train class	Train sampler	Primary metric
TrashNet	6	1,768	379	380	trash 96 / paper 416	class-balanced (weighted)	macro-F1
RPS-25	3	630	33	372	210 / 210 (balanced)	shuffle	top-1 accuracy

(Table III). The best validation checkpoint is restored. Two cells depart from this. One `scratch` seed on RPS-25/LeNet-5 used a different budget, and the `kd` cell for that pair mixes two temperatures. Checkpoints are selected by validation macro-F1 on TrashNet. On RPS-25 they are selected by validation loss, because the 33-image validation split makes validation F1 too erratic to select on. That validation loss is each method's own training objective evaluated on the validation batch. For the kernel methods it includes the kernel term at $\gamma_{\max}$, computed on a single 33-image Gram matrix. Each cell is trained with seeds 0, 1 and 2, which set the student initialization, the data order and the random teacher projection. Table IV lists the loss settings that were used.

D. Selection, reporting and the status of the results

Every number is the mean ± sample standard deviation ($n-1$) over three seeds. Comparisons are paired by seed. We report the mean paired difference, its standard deviation and the number of seeds on which the first method is ahead. Two-sided paired t-tests are given as descriptive summaries and are not corrected for multiple comparisons. For the follow-up arms (`pqk_adaptive`, `noent_control`, `pqk_fit`, `rbf_fit`) we checked that a shared seed means identical student initialization and data order, because the first-batch cross-entropy and KL gradients are identical across the arms. The stored gradient logs confirm this for `noent_control`, `pqk_fit` and `rbf_fit` in every cell. For `pqk_adaptive` the check was made when the arms were set up, and its logs were stored for two runs only. For the earlier grid, pairing

TABLE III

STUDENT RECIPES. BN: BATCH-NORMALIZATION STATISTICS ARE EITHER THE USUAL RUNNING ESTIMATES OR THE PER-BATCH STATISTICS USED DURING TRAINING WITH A RECALIBRATION PASS ON CLEAN TRAINING IMAGES BEFORE EVERY EVALUATION (NEEDED WHEN AN EPOCH HAS TOO FEW BATCHES, OR FOR MOBILENETV3-SMALL'S HARDSWISH AND SQUEEZE-EXCITE BLOCKS, TO KEEP RUNNING STATISTICS STABLE). RRC: MINIMUM SCALE OF THE RANDOM RESIZED CROP. *ONE OF THE THREE SEEDS USED A DIFFERENT BUDGET.

Dataset	Student	Epochs	Patience	LR	BN stats	RRC min scale	RandAugment (ops, mag.)
TrashNet	MobileNetV2-0.35	60	10	0.001	running	0.85	1, 5
TrashNet	MobileNetV3-S	40	8	0.001	per-batch + recal.	0.85	1, 5
TrashNet	LeNet-5	60	12	0.001	running	0.85	1, 5
RPS-25	MobileNetV2-0.35	60	10	0.001	per-batch + recal.	0.7	2, 9
RPS-25	MobileNetV3-S	40	8	0.001	per-batch + recal.	0.7	2, 9
RPS-25	LeNet-5	40/60*	8/12	0.001	per-batch + recal.	0.7	2, 9

TABLE IV

LOSS SETTINGS RECORDED IN THE RUNS. KERNEL METHODS SHARE $\tau = 0.5$, WHEREAS THE `KD` TEMPERATURE WAS SCREENED ON SOME PAIRS AND REUSED ON OTHERS (SEE TEXT). ¶THE SEEDS OF THIS CELL USED DIFFERENT SETTINGS.

Method	Loss setting	Pairs
`scratch`	CE×1	all pairs
`kd`	α=0.5, β=0.5, τ=3	TrashNet (all students)
	α=0.5, β=0.5, τ=1	RPS-25: MobileNetV2-0.35, MobileNetV3-S
	α=0.5, β=0.5, τ=0.5	RPS-25: LeNet-5 (seed 0)¶
	α=0.5, β=0.5, τ=1	RPS-25: LeNet-5 (seeds 1, 2)¶
`rkd`	CE×1, dist.×5, angle×10	TrashNet: MobileNetV2-0.35, MobileNetV3-S; RPS-25: MobileNetV2-0.35, MobileNetV3-S
	CE×1, dist.×1, angle×2	TrashNet: LeNet-5
`rbf_control`	α=0.5, β=0.5, τ=0.5, γ=1 (10-epoch ramp)	all pairs
`pqk`	α=0.5, β=0.5, τ=0.5, γ=1 (10-epoch ramp)	all pairs
`pqk_adaptive`	α=0.5, β=0.5, τ=0.5, γ=1 (10-epoch ramp)	all pairs
`noent_control`	α=0.5, β=0.5, τ=0.5, γ=1 (10-epoch ramp)	TrashNet: MobileNetV2-0.35, MobileNetV3-S; RPS-25: MobileNetV2-0.35, MobileNetV3-S

means a shared seed value only.

These results are exploratory, for three reasons. First, several design choices were screened on seed 0, and some were judged by test-split scores. These are the student recipes (sampler, augmentation strength and epoch budget, tuned on MobileNetV2 for TrashNet), the KD temperature, and the RKD weights. Second, the temperature of the kernel methods ($\tau = 0.5$) was inherited from a KD screen on LeNet-5 with RPS-25 and was not re-tuned per pair. The `kd` temperature was screened on TrashNet/MobileNetV2-0.35 ($\tau = 3$) and RPS-25/MobileNetV2-0.35 ($\tau = 1$) and reused for the other students of the dataset (RPS-25/LeNet-5 is a special case). Comparisons between a kernel method and `kd` are therefore not matched on that parameter, and may disadvantage the kernel methods. Third, the choice of λ for `pqk` was made after results were seen. The pre-specified $\lambda = 1$ gave the `pqk` results of the first full grid. The adaptive λ was adopted afterwards, per pair, by the higher three-seed *validation* macro-F1 of the two variants. For the choice of λ and for the follow-up analyses, validation scores decide and test scores are descriptive. The headline comparisons show both λ variants. A confirmatory claim would need a frozen protocol, selection on validation alone and fresh data, which we do not have.

E. INT8 export

Each trained MobileNet student is exported in three steps. The projection and kernel modules are stripped, the PyTorch graph is converted to ONNX (opset 17, static batch size 1), and ONNX is converted to a fully integer TFLite model with per-channel weight quantization. Calibration uses 256 unaugmented training images, and input and output are quantized to INT8. Four checks run on every model: a purity check that the graph contains no projection, quantum or custom operations; a check that every convolution is quantized per channel and the whole graph is integer; ONNX–PyTorch parity on the logits; and reload of the fp32 weights reproducing the recorded test score. The INT8 test score is measured with the LiteRT interpreter on an x86 PC, whose default and builtin kernels give identical outputs ("PC" below). The exported set contains 66 models: two students, two datasets, five methods and three seeds, plus the second λ variant of `pqk` on the two pairs where both were exported. For `pqk` the exported variant is the one the validation rule had chosen at export time. For TrashNet with MobileNetV2 that was $\lambda = 1$, whereas the final rule prefers adaptive λ, which has no INT8 export (marked ‡ in the tables).

F. On-device benchmark

The INT8 models run on a Raspberry Pi 4 Model B (Cortex-A72, four cores, 4 GB, Ubuntu 20.04, `ondemand` governor, 600–1800 MHz) with LiteRT (`ai-edge-litert` 2.1.0, Python 3.13). Each model runs in a fresh process with 50 warm-up and 500 timed invocations at batch size 1, and every run starts below 55 °C. We log invocation latency percentiles, the full quantize–invoke–dequantize pipeline, load and cold-start time, CPU use, resident memory, and SoC temperature and clock every 100 ms. We also read the firmware's throttling flags before and after each timed window. The suite covers all models with the default XNNPACK delegate and with the

builtin kernels only, a thread sweep from one to four threads, and fp32 references. Each model is also scored on the full test split on the device and compared with the PC's logits. A live USB-camera pipeline (640×480, MJPEG) measures the end-to-end cost including image preparation. Energy was not measured because no power meter was attached, and the Jetson Orin Nano and ESP32-S3 targets were not benchmarked.

G. Compute

All training ran on a single GPU, with several runs often sharing it. Per-epoch wall-clock times are therefore not comparable across methods, and we report compute-hours instead (Section V-G). The 152 student runs in the results include, besides the grid, the follow-up arms, the screens and seven runs of two wider MobileNetV2 variants (width multipliers 0.5 and 0.6). Those seven were stopped as incomplete capacity probes and are not reported.

V. Results

A. Distillation baselines

Table V and Figure 4 give the baseline picture. Distillation helps in some pairs and not in others. `kd` beats `scratch` on TrashNet for both MobileNets, by $+0.025 \pm 0.008$ (MobileNetV2-0.35) and $+0.009 \pm 0.007$ (MobileNetV3-Small) macro-F1, ahead on all three seeds of each. On the 0.03 M LeNet-5 it hurts (-0.045 ± 0.093), and `rkd` helps ($+0.049 \pm 0.035$, three of three seeds). `rkd` is slightly behind `scratch` on the TrashNet MobileNets (-0.012 and -0.008). No method dominates. The best mean is `kd` on the TrashNet MobileNets, `rkd` on TrashNet/LeNet-5 and RPS-25/MobileNetV2-0.35, `rbf_control` on RPS-25/MobileNetV3-Small, and `scratch` on RPS-25/LeNet-5. `pqk` is the best method on no pair. The classical kernel-alignment arm `rbf_control` beats `scratch` on the mean in only 2 of 6 pairs. On RPS-25/MobileNetV3-Small it does so clearly ($+0.072 \pm 0.034$, three of three seeds). On TrashNet all students stay well below the teacher: the best reaches 0.732 macro-F1 against 0.876. On RPS-25 the best student (the RBF twin with MobileNetV3-Small, 0.888) comes close to the teacher's 0.907. Seed noise is large relative to these effects, particularly on RPS-25, where the standard deviation across seeds ranges from 0.012 to 0.202, against 0.003 to 0.074 on TrashNet.

B. PQK against logit KD, the RBF twin and relational KD

a) Against logit KD.: Table VI and Figure 5 show that `pqk` and `kd` cannot be told apart. At fp32, `pqk` is ahead of `kd` on the mean in 3 of 6 pairs. The count is 4 if the one `kd` seed trained at a different temperature on RPS-25/LeNet-5 is excluded. On TrashNet, the primary dataset, `pqk` is behind on both MobileNets, by 0.025 ± 0.025 and 0.014 ± 0.021 macro-F1, and ahead on one seed of three in each. It is ahead on TrashNet/LeNet-5 ($+0.051 \pm 0.073$), where `kd` itself fails. On RPS-25 it is ahead by $+0.003 \pm 0.073$ (MobileNetV2-0.35) and $+0.010 \pm 0.112$ (MobileNetV3-Small) and behind on LeNet-5 (-0.013 ± 0.059). On top-1, the primary metric there, the three differences are $+0.012$, $+0.008$ and -0.010. After INT8 conversion, `pqk` is ahead of `kd` on the mean in 3 of 4 pairs on PC numerics and on the Pi's builtin kernels, and in 2 of 4 on the Pi's default XNNPACK kernels. TrashNet/MobileNetV3-Small is a loss under all three sets of kernels (-0.017, -0.016, -0.025, with `pqk` behind on every seed). The RPS-25 differences ($+0.011$ to $+0.032$) come with paired standard deviations of 0.10 to 0.12. No difference is significant. The smallest uncorrected paired p-value is 0.22 at fp32 and 0.10 at INT8. One caveat applies: the `kd` temperature was screened on some pairs, whereas the kernel methods use one untuned τ (Section IV-D). This comparison is therefore not matched on that parameter. Against `scratch`, `pqk` differs by between -0.039 and $+0.024$ across the six pairs, with a smallest p-value of 0.37.

b) Against the RBF twin.: At the pre-specified $\lambda = 1$, `pqk` is ahead of its twin on the mean in only 2 of 6 pairs (Table VII, Figure 6). These are TrashNet/MobileNetV2-0.35 ($+0.023 \pm 0.016$, three of three seeds) and RPS-25/LeNet-5 ($+0.100 \pm 0.145$, two of three). It is behind on the other four (-0.042, -0.004, -0.028, -0.048). The two pairs it wins are the two that were evaluated first. Four further pairs did not reproduce them. With the adaptive λ, which gives `pqk` the same calibration as the twin, `pqk` is ahead on 3 of 4 pairs ($+0.013 \pm 0.011$, $+0.003 \pm 0.036$, $+0.029 \pm 0.088$) and behind on RPS-25/MobileNetV3-Small (-0.058 ± 0.011). It wins on 3, 2, 1 and 0 seeds of three respectively, so the second and third winning means rest on one or two seeds. The smallest p-value for $\lambda = 1$ is 0.13. For the adaptive variant it is 0.01, on RPS-25/MobileNetV3-Small, where the difference favours the classical twin. The quantum kernel shows no consistent advantage over its classical counterpart.

c) Against relational KD.: QRKD is a quantum extension of RKD, so `rkd` is the classical ancestor of `pqk` (Table VIII). `pqk` is ahead of `rkd` on the two TrashNet MobileNet pairs ($+0.012 \pm 0.016$ and $+0.003 \pm 0.008$). It is behind on TrashNet/LeNet-5 (-0.043 ± 0.018, behind on every seed, $p = 0.052$), RPS-25/MobileNetV2-0.35 (-0.024 ± 0.023) and RPS-25/MobileNetV3-Small (-0.009 ± 0.111). That is 2 of 5 pairs, by at most 0.012 macro-F1. The setting differs from that of QRKD, so this is not a failed replication. We do not observe consistent gains over classical relational distillation.

C. Seed-to-seed variance

The clearest pattern in favour of `pqk` is in the spread across seeds (Table IX, Figure 7), and it is weaker than it first looks. On macro-F1, with the validation-selected λ, its standard deviation is lower than that of `rbf_control` on 4 of 6 pairs, lower than that of `scratch` on 4, and lower than both on three. On top-1 accuracy the counts are 5 and 5. Pooled over pairs (root mean square), it is 0.041 against 0.092 for `rbf_control` and 0.052 for `scratch`. The `scratch` figure includes the one recipe-mismatched run on RPS-25/LeNet-5. Without that seed the pooled `scratch` value is 0.032, below that of `pqk`. The advantage over `rbf_control` is the more consistent one. Against `scratch` it is fragile. Four things limit what the pattern supports. *First*, a standard deviation from three seeds

TABLE V

TEST MACRO-F1 AND TOP-1 ACCURACY, MEAN ± SAMPLE STD OVER THREE SEEDS. BOLD: BEST MEAN IN A COLUMN WITHIN A STUDENT. †PQK WITH THE VALIDATION-SELECTED ADAPTIVE λ (ELSEWHERE $\lambda = 1$). §ONE OF THE THREE SEEDS USED A DIFFERENT TRAINING BUDGET, AND ¶THE SEEDS OF THE KD CELL USED TWO DIFFERENT TEMPERATURES. RKD WAS NOT PART OF THE FINAL GRID FOR RPS-25/LENET-5. THE PRIMARY METRIC IS MACRO-F1 ON TRASHNET AND TOP-1 ON RPS-25.

		TrashNet (6 classes)		RPS-25 (3 classes)	
Student	Method	Macro-F1	Top-1	Macro-F1	Top-1
MobileNetV2-0.35	scratch	0.684 ± 0.014	0.697 ± 0.021	0.710 ± 0.023	0.727 ± 0.030
	kd	**0.708** ± 0.013	**0.725** ± 0.009	0.710 ± 0.088	0.719 ± 0.087
	rkd	0.672 ± 0.004	0.686 ± 0.003	**0.737** ± 0.051	**0.745** ± 0.048
	rbf_control	0.671 ± 0.010	0.692 ± 0.012	0.684 ± 0.062	0.702 ± 0.059
	pqk	0.683 ± 0.013†	0.694 ± 0.016†	0.713 ± 0.030†	0.730 ± 0.019†
MobileNetV3-S	scratch	0.724 ± 0.016	0.745 ± 0.016	0.816 ± 0.053	0.828 ± 0.045
	kd	**0.732** ± 0.020	**0.753** ± 0.021	0.830 ± 0.089	0.843 ± 0.075
	rkd	0.716 ± 0.003	0.736 ± 0.007	0.849 ± 0.048	0.859 ± 0.041
	rbf_control	0.715 ± 0.037	0.730 ± 0.040	**0.888** ± 0.063	**0.892** ± 0.060
	pqk	0.718 ± 0.009†	0.735 ± 0.008†	0.840 ± 0.065	0.851 ± 0.054
LeNet-5	scratch	0.529 ± 0.021	0.541 ± 0.022	**0.676** ± 0.108§	**0.676** ± 0.109§
	kd	0.484 ± 0.074	0.490 ± 0.075	0.650 ± 0.012¶	0.652 ± 0.016¶
	rkd	**0.578** ± 0.016	**0.597** ± 0.021	–	–
	rbf_control	0.539 ± 0.008	0.552 ± 0.007	0.538 ± 0.202	0.584 ± 0.153
	pqk	0.535 ± 0.003	0.552 ± 0.003	0.638 ± 0.070	0.642 ± 0.058

TABLE VI

PAIRED DIFFERENCE pqk − kd IN TEST MACRO-F1 (MEAN ± STD OF THE PER-SEED DIFFERENCES, AND THE NUMBER OF SEEDS ON WHICH PQK IS AHEAD). PQK IS AT ITS VALIDATION-SELECTED λ AT FP32 AND AT THE EXPORTED VARIANT AT INT8; THESE DIFFER ONLY FOR TRASHNET/MOBILENETV2-0.35, WHERE THE INT8 MODELS USE $\lambda = 1$ AND THE FP32 COLUMN ADAPTIVE λ. ¶THE KD CELL FOR RPS-25/LENET-5 MIXES TWO TEMPERATURES. "PC" USES LITERT'S BUILTIN KERNELS ON X86; "PI" COLUMNS WERE MEASURED ON THE RASPBERRY PI, WITH ITS BUILTIN KERNELS AND WITH ITS DEFAULT XNNPACK DELEGATE. LENET-5 WAS NOT EXPORTED.

		fp32	INT8		
Dataset	Student	test macro-F1	PC (builtin)	Pi builtin	Pi XNNPACK
TrashNet	MobileNetV2-0.35	−0.025 ± 0.025 (1/3)	+0.004 ± 0.006 (2/3)	+0.004 ± 0.006 (2/3)	−0.004 ± 0.010 (1/3)
TrashNet	MobileNetV3-S	−0.014 ± 0.021 (1/3)	−0.017 ± 0.021 (0/3)	−0.016 ± 0.016 (0/3)	−0.025 ± 0.015 (0/3)
TrashNet	LeNet-5	+0.051 ± 0.073 (2/3)	–	–	–
RPS-25	MobileNetV2-0.35	+0.003 ± 0.073 (2/3)	+0.025 ± 0.108 (2/3)	+0.025 ± 0.108 (2/3)	+0.032 ± 0.113 (2/3)
RPS-25	MobileNetV3-S	+0.010 ± 0.112 (2/3)	+0.016 ± 0.101 (2/3)	+0.017 ± 0.104 (2/3)	+0.011 ± 0.115 (2/3)
RPS-25	LeNet-5	−0.013 ± 0.059 (2/3)¶	–	–	–
Pairs where pqk is ahead on the mean		3/6	3/4	3/4	2/4

is itself very uncertain, and a sign test on four of six pairs gives $p = 0.69$. *Second*, the pattern depends on the calibration: with $\lambda = 1$, pqk is below scratch on three pairs and below rbf_control on five. *Third*, it depends on the metric and on the mismatched run. Without that seed, the count against scratch is 3 of 6 on macro-F1 and 4 on top-1. *Fourth*, pqk is not the most stable method. On the five pairs where rkd exists, the pooled standard deviations are 0.029 for scratch, 0.032 for rkd and 0.033 for pqk, and rkd is below pqk on three of the five. After INT8 conversion the pattern weakens further. Pooled over the four exported pairs on PC numerics, pqk has a standard deviation of 0.042 against 0.037 for scratch and 0.025 for rbf_control, because RPS-25/MobileNetV2-0.35 is noisier for pqk. On TrashNet, pqk remains below both on both students under all three sets of kernels.

D. Entanglement and the teacher projection

a) Removing the entanglement.: On TrashNet, whose validation split (379 images) is far larger than that of RPS-25, pqk is ahead of its entanglement-free twin on every seed of both students (validation +0.049 ± 0.065 and +0.007 ± 0.008, Table X). This is the one consistent sign that the CNOT gates contribute something beyond the sixteen trigonometric coordinates. On RPS-25 the sign reverses (−0.041 and −0.022), on a 33-image validation set. Test scores point the same way except on TrashNet/MobileNetV3-Small, where the difference vanishes (−0.002, one seed of three). The claim is limited. The entanglement-free readout $(\sin 4\theta, \cos 4\theta)$ is highly oscillatory over $\theta \in (0, \pi)$ and may simply be a poor classical map. This result therefore does not show that pqk beats every classical map of equal width. The control we designed for that purpose, random Fourier features on the circuit's own frequency spectrum, was not run. Nor did the class separation of the target kernel predict the outcome. The entanglement-free target is better aligned with the labels than the PQK target on TrashNet (0.323 against 0.245 centered alignment on validation), yet students trained on the PQK target rank higher.

b) A fitted teacher projection.: The fixed random 2048 → 8 projection discards much of the teacher's class structure. The centered alignment of the teacher kernel with the label kernel falls from 0.720 for the raw features to 0.363 for the random

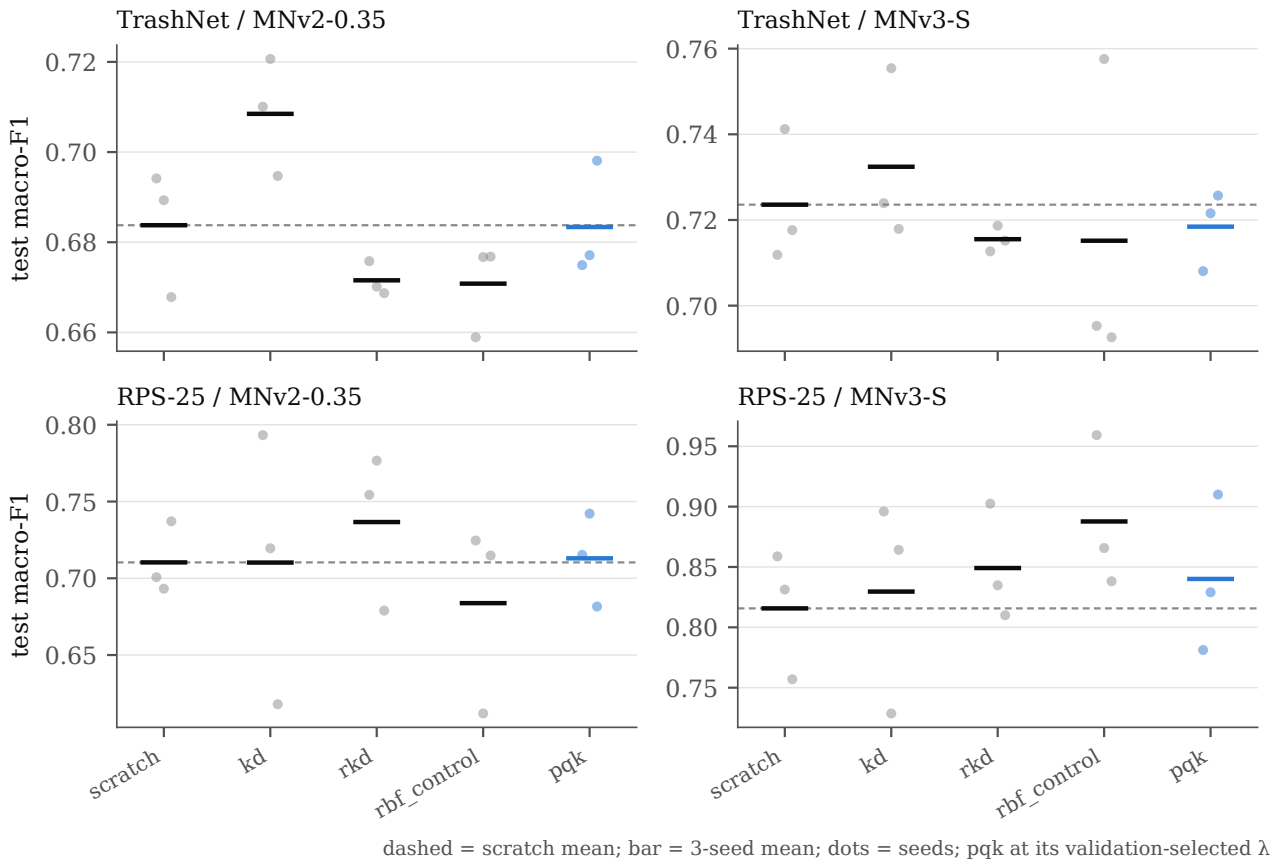


Fig. 4. Test macro-F1 of the five methods for the four MobileNet pairs. Dots are individual seeds, bars are means, and the dashed line is the mean of `scratch`.

TABLE VII

TEST MACRO-F1 OF THE TWIN `RBF_CONTROL` AND THE PAIRED DIFFERENCE `pqk` − `rbf_control` FOR THE TWO λ CALIBRATIONS OF `PQK`. ADAPTIVE λ HAS ONLY ONE SEED ON THE TWO LENET-5 PAIRS AND IS OMITTED THERE. [§]SEE TABLE V.

Dataset	Student	`scratch`	`rbf_control`	Δ `pqk` (λ=1)	Δ `pqk` (adaptive λ)	selected
TrashNet	MobileNetV2-0.35	0.684 ± 0.014	0.671 ± 0.010	+0.023 ± 0.016 (3/3)	+0.013 ± 0.011 (3/3)	adaptive
TrashNet	MobileNetV3-S	0.724 ± 0.016	0.715 ± 0.037	−0.042 ± 0.049 (1/3)	+0.003 ± 0.036 (2/3)	adaptive
TrashNet	LeNet-5	0.529 ± 0.021	0.539 ± 0.008	−0.004 ± 0.010 (2/3)	–	λ=1
RPS-25	MobileNetV2-0.35	0.710 ± 0.023	0.684 ± 0.062	−0.028 ± 0.082 (1/3)	+0.029 ± 0.088 (1/3)	adaptive
RPS-25	MobileNetV3-S	0.816 ± 0.053	0.888 ± 0.063	−0.048 ± 0.125 (1/3)	−0.058 ± 0.011 (0/3)	λ=1
RPS-25	LeNet-5	0.676 ± 0.108[§]	0.538 ± 0.202	+0.100 ± 0.145 (2/3)	–	λ=1
Pairs where `pqk` beats its twin on the 3-seed mean					2/6 (λ=1); 3/4 (adaptive)	

angles on TrashNet validation. A projection fitted on the training split (LDA+PCA) restores it (0.825). We selected it over PCA on validation alignment before any student was trained. It did not help. Fitted was better than random in *one of eight* arm-pairs on validation (Table XI, Figure 8a). The exception is `pqk` on RPS-25/MobileNetV3-Small (+0.043 ± 0.039). The alignment measured on training batches suggests why on TrashNet. There the fitted target is almost the label kernel on the data the loss sees: 0.963 for the RBF target on training batches against 0.825 on validation (random: 0.447), and 0.902 against 0.752 for the PQK target (random: 0.297 against 0.245). Aligning to it asks the student to cluster by class, which cross-entropy already does, and it removes the within-class geometry that relational distillation exists to transfer. On RPS-25 the picture is mixed, and this account does not explain it. The fitted RBF target is not saturated on training batches (0.844 against 0.861 on validation). The fitted PQK target is more aligned on training batches than on validation (0.897 against 0.797), and that arm produced the one positive case. The saturation account therefore covers the TrashNet results only. It is a hypothesis supported by the measurement, and we did not test it by intervention. The measurement also shows that the selection criterion, validation alignment, was the wrong one for a relational target, because it rewards class separation.

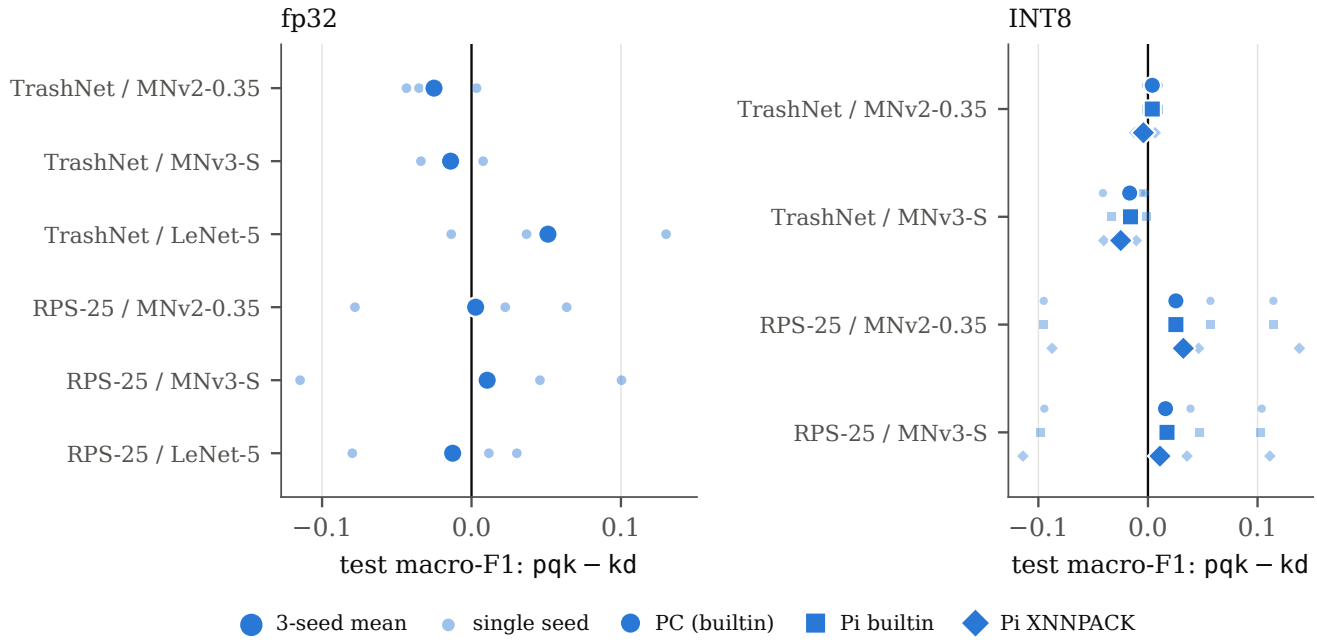


Fig. 5. `pqk` − `kd` per pair, paired by seed, at fp32 (left) and after INT8 conversion under three sets of kernels (right). Positive values favour `pqk`.

TABLE VIII
PAIRED DIFFERENCE `pqk` − `rkd` IN TEST MACRO-F1. `RKD` WAS NOT PART OF THE FINAL GRID FOR RPS-25/LENET-5.

Dataset	Student	`rkd`	Δ `pqk` − `rkd`
TrashNet	MobileNetV2-0.35	0.672 ± 0.004	+0.012 ± 0.016 (2/3)
TrashNet	MobileNetV3-S	0.716 ± 0.003	+0.003 ± 0.008 (2/3)
TrashNet	LeNet-5	0.578 ± 0.016	−0.043 ± 0.018 (0/3)
RPS-25	MobileNetV2-0.35	0.737 ± 0.051	−0.024 ± 0.023 (1/3)
RPS-25	MobileNetV3-S	0.849 ± 0.048	−0.009 ± 0.111 (1/3)
RPS-25	LeNet-5	–	–

c) Does the kernel term dominate training?: Not with the random target. For the entanglement-free arm (12 runs), the ratio of the kernel gradient to the cross-entropy gradient falls from a mean of 0.70 over the first three epochs to 0.14 over the last five. For `pqk`, which was logged for only two runs (TrashNet/MobileNetV2-0.35; Figure 9), it falls from 0.88 to 0.20. The label and logit terms therefore drive training after the first epochs. With a fitted target the ratio over the last five epochs is 0.74, 0.62 and 0.34 for `pqk` and 0.45, 0.30 and 0.28 for `rbf_control` on TrashNet/MobileNetV2-0.35, TrashNet/MobileNetV3-Small and RPS-25/MobileNetV3-Small. On RPS-25/MobileNetV2-0.35 it is 3.1 and 2.1, and the kernel term outweighs the label signal.

E. INT8 conversion

All 66 exported models pass every check. They are fully integer, quantized per channel on every convolution, contain no custom or floating-point operations, and reproduce their fp32 score exactly on reload. Their ONNX graphs match the PyTorch logits to within 1.3×10^{-4}. Their size is set by the architecture alone: 0.62 MB (604–608 KiB) for MobileNetV2-0.35 and 1.21 MB (1179–1181 KiB) for MobileNetV3-Small. Any comparison at matched size therefore holds by construction. Conversion has no systematic cost. The macro-F1 changes by +0.003 ± 0.022 across the 66 models, and 37 of them improve. Conversion does change predictions. The INT8 and fp32 models disagree on about 7% of test images (91.5% agreement for MobileNetV2-0.35 and 95.3% for MobileNetV3-Small), in both directions. INT8 rankings between methods are therefore fp32 rankings plus noise. `pqk` has the most favourable change of the five methods on PC numerics (+0.0080, up on 9 of 12 models, against −0.0031 and 5 of 12 for `kd`). The difference between the two changes is not significant ($p = 0.22$ on PC, 0.19 on the Pi's builtin kernels, 0.49 with XNNPACK), and it shrinks to +0.0025 with the Pi's default kernels (Table XII, Figure 10). The sign of other methods' changes also depends on the kernels: `rbf_control` moves from −0.0064 on PC to +0.0006 with XNNPACK, and `rkd` from +0.0030 to −0.0011.

F. On-device behaviour

Table XIII and Figure 11 summarize the measurements (160 runs, none failed). With four threads the median INT8 latency

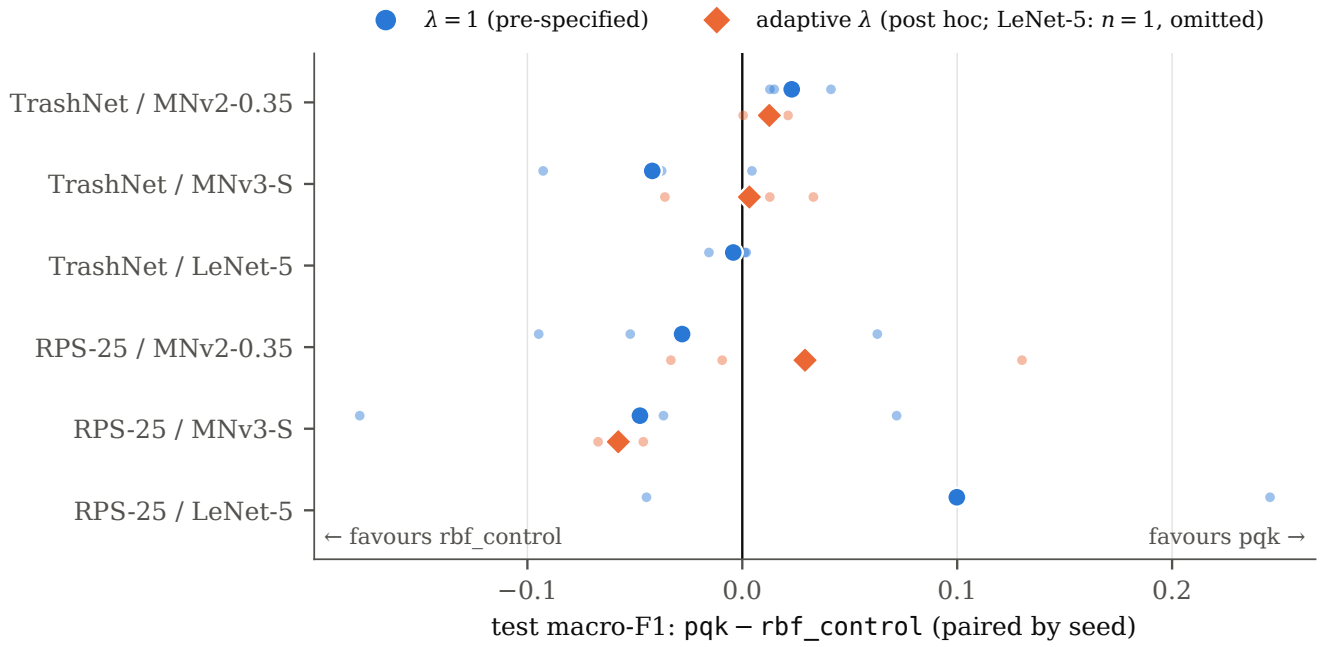


Fig. 6. `pqk` − `rbf_control` per pair and seed, for the pre-specified $\lambda = 1$ and for the adaptive λ adopted after the first results. Large markers are three-seed means.

TABLE IX

STANDARD DEVIATION OF TEST MACRO-F1 ACROSS THE THREE SEEDS. BOLD: LOWEST OF THE FIVE METHODS, WITH `PQK` AT ITS VALIDATION-SELECTED λ (LAST COLUMN); THE TWO FIXED AND ADAPTIVE COLUMNS SHOW BOTH CALIBRATIONS. THE LAST ROW IS THE ROOT MEAN SQUARE OVER PAIRS (ADAPTIVE: FOUR PAIRS, `RKD`: FIVE). [§],[¶]SEE TABLE V.

Dataset	Student	`scratch`	`kd`	`rkd`	`rbf_control`	`pqk` λ=1	`pqk` adaptive	`pqk` selected
TrashNet	MobileNetV2-0.35	0.014	0.013	**0.004**	0.010	0.006	0.013	0.013
TrashNet	MobileNetV3-S	0.016	0.020	**0.003**	0.037	0.023	0.009	0.009
TrashNet	LeNet-5	0.021	0.074	0.016	0.008	0.003	–	**0.003**
RPS-25	MobileNetV2-0.35	**0.023**	0.088	0.051	0.062	0.031	0.030	0.030
RPS-25	MobileNetV3-S	0.053	0.089	**0.048**	0.063	0.065	0.074	0.065
RPS-25	LeNet-5	0.108[§]	**0.012**[¶]	–	0.202	0.070	–	0.070
Pooled std (root-mean-square over pairs)		0.052	0.060	0.032	0.092	0.042	0.041	0.041

is 2.5 ms for MobileNetV2-0.35 and 3.2 ms for MobileNetV3-Small, about 395 and 315 inferences per second. Latency is a property of the architecture. Within a pair, the spread between methods, which share one graph, is at most 0.11 ms. The two networks do about the same work (18.9 and 18.2 M multiply–accumulates), yet MobileNetV2 is about 25% faster on four threads, so operation counts do not predict speed. MobileNetV2 also scales better with threads (1.75–1.84× from one to four, against 1.35–1.40×), and on one thread the order reverses (4.56 against 4.35 ms on TrashNet). Against fp32 with the same runtime (one model per pair), INT8 is 1.2 to 1.4× faster, 2.6 to 3.1× smaller, and uses less memory (5.4–6.7 MB of interpreter memory against 8–12 MB). The speed-up is modest, and the Cortex-A72's lack of integer dot-product instructions is a likely reason. XNNPACK is 1.1 to 1.4× faster than the builtin kernels at four threads, but it changes the model's predictions. Per model it disagrees with the PC's INT8 predictions on 0.3% to 9.4% of test images (3.5% on average), against at most 2.4% (0.6% on average) for the builtin kernels. It moves per-method macro-F1 by between -0.016 and +0.012 relative to the PC. It is also the reason the `pqk`–`kd` comparison on TrashNet/MobileNetV2-0.35 changes sign on the device (+0.004 on PC, −0.004 with XNNPACK; Table VI). INT8 results should therefore be reported together with the runtime that produced them. The SoC reached at most 64.8 °C, well below the 80 °C throttling point. The firmware did flag an under-voltage event and a throttling event in 2 of the 160 runs, both at four threads, so the supply used here is marginal, and we did not attempt an energy measurement. A live-camera pipeline reaches about 30 frames per second, the webcam's rate, with two inference and two OpenCV threads at 12.8 ms of processing per frame. Image preparation (8.8 ms) costs more than inference (4.1 ms). The 2.5 and 3.2 ms above are invocation times only; the quantize–invoke–dequantize pipeline takes 3.4 to 4.1 ms.

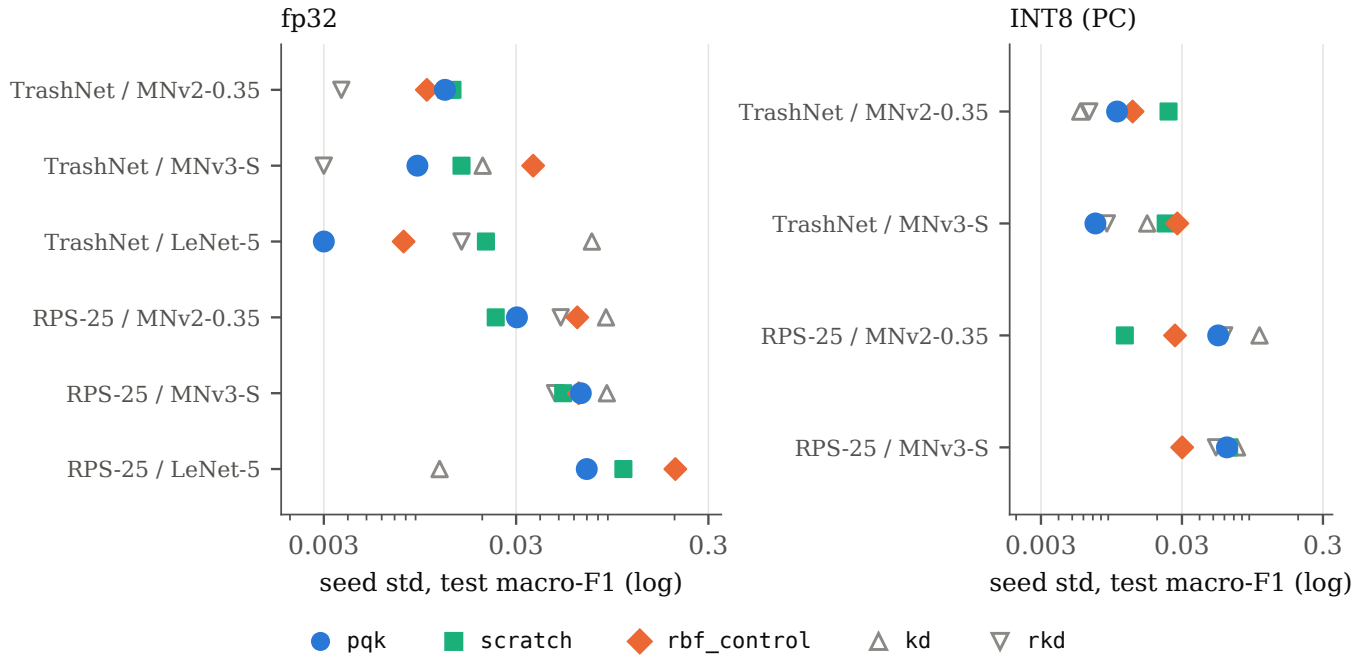


Fig. 7. Seed standard deviation of test macro-F1 per method and pair on a log scale, at fp32 (left) and after INT8 conversion on PC (right). `pqk` is at its validation-selected λ at fp32 and at the exported variant at INT8.

TABLE X

`PQK` (ADAPTIVE λ) AGAINST THE SAME CIRCUIT WITH ITS CNOT GATES REMOVED. VALIDATION (DECISIVE) AND TEST (DESCRIPTIVE) MACRO-F1, PAIRED BY SEED; Δ IS `pqk` − `noent_control`. VALIDATION HAS 379 IMAGES ON TRASHNET AND 33 ON RPS-25.

Dataset	Student	`pqk`	no-CNOT	Δ val	Δ test
TrashNet	MobileNetV2-0.35	0.705 ± 0.006	0.655 ± 0.060	+0.049 ± 0.065 (3/3)	+0.059 ± 0.079 (3/3)
TrashNet	MobileNetV3-S	0.737 ± 0.003	0.729 ± 0.006	+0.007 ± 0.008 (3/3)	−0.002 ± 0.028 (1/3)
RPS-25	MobileNetV2-0.35	0.797 ± 0.083	0.839 ± 0.038	−0.041 ± 0.044 (0/3)	−0.060 ± 0.035 (0/3)
RPS-25	MobileNetV3-S	0.874 ± 0.057	0.896 ± 0.037	−0.022 ± 0.078 (1/3)	−0.025 ± 0.105 (1/3)

TABLE XI

FITTED MINUS RANDOM TEACHER PROJECTION, PAIRED BY SEED: Δ VALIDATION AND TEST MACRO-F1 FOR `PQK` (ADAPTIVE λ) AND `RBF_CONTROL`. NEGATIVE VALUES FAVOUR THE RANDOM PROJECTION.

		`pqk`		`rbf_control`	
Dataset	Student	Δ val	Δ test	Δ val	Δ test
TrashNet	MobileNetV2-0.35	−0.049 ± 0.041 (0/3)	−0.060 ± 0.073 (1/3)	−0.009 ± 0.016 (1/3)	−0.018 ± 0.003 (0/3)
TrashNet	MobileNetV3-S	−0.001 ± 0.020 (1/3)	−0.014 ± 0.019 (1/3)	−0.037 ± 0.012 (0/3)	−0.013 ± 0.049 (1/3)
RPS-25	MobileNetV2-0.35	−0.087 ± 0.090 (0/3)	−0.104 ± 0.035 (0/3)	−0.002 ± 0.050 (1/3)	−0.091 ± 0.125 (1/3)
RPS-25	MobileNetV3-S	+0.043 ± 0.039 (2/3)	+0.029 ± 0.133 (2/3)	−0.022 ± 0.036 (0/3)	−0.053 ± 0.070 (1/3)

G. Training cost

The 152 student runs account for 95 compute-hours. Of these, 45 belong to the 54 runs of the `pqk` family, and a further 23 hours went into runs discarded after pipeline fixes (Table XIV). Because runs shared the GPU, we cannot give a clean cost ratio between methods. The accounting does show that the circuit adds simulation to every training step of a `pqk` run, while nothing of it remains at deployment, where all methods give models of identical size and speed.

VI. CONCLUSION

We asked whether a quantum kernel used only during training helps a lightweight waste classifier beyond what a classical kernel in the same place provides. We tested this on TrashNet, with RPS-25 as a low-data confirmation, using three students and five methods under one pipeline. A projected quantum kernel was not distinguishable from logit KD in accuracy. It was ahead on the mean in 3 of 6 pairs and behind on both TrashNet MobileNets, and no paired difference was significant. It beat its RBF twin in 2 of 6 pairs at its pre-specified scale. It also

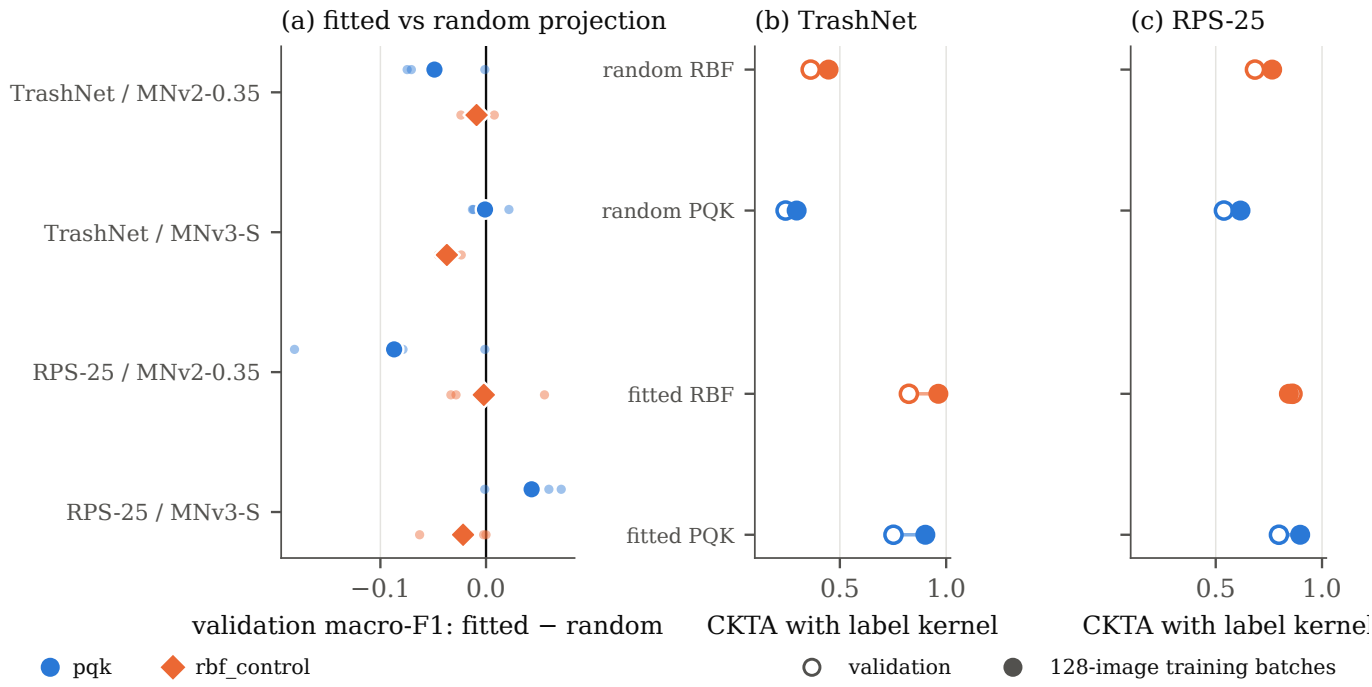


Fig. 8. (a) Validation macro-F1 of a fitted (LDA+PCA) minus a random teacher projection, per seed. (b,c) Centered kernel–target alignment (CKTA) of the target kernel the student is asked to match with the label kernel, on validation (open) and on 128-image training batches (filled), where the loss is computed.

TABLE XII
CHANGE IN TEST MACRO-F1 FROM FP32 TO INT8, POOLED OVER THE EXPORTED PAIRS AND SEEDS (MEAN, AND THE NUMBER OF THE 12 MODELS WHOSE SCORE ROSE), FOR EACH METHOD AND SET OF KERNELS.

Method	PC (builtin)	Pi builtin	Pi XNNPACK
`scratch`	+0.0058 (7/12)	+0.0038 (7/12)	+0.0011 (5/12)
`kd`	−0.0031 (5/12)	−0.0027 (5/12)	−0.0049 (4/12)
`rkd`	+0.0030 (6/12)	+0.0033 (6/12)	−0.0011 (5/12)
`rbf_control`	−0.0064 (5/12)	−0.0045 (5/12)	+0.0006 (6/12)
`pqk`	+0.0080 (9/12)	+0.0088 (9/12)	+0.0025 (5/12)

showed a tendency to lower seed-to-seed variance than the twin and, less consistently, than hard-label training. That tendency is not significant at three seeds, depends on the metric and on one mismatched run, and does not hold on every pair. Removing the circuit's entanglement hurt on TrashNet validation and helped on RPS-25, and fitting the teacher projection did not help. All 66 exported MobileNet students are fully integer INT8 models of about 0.62 and 1.21 MB. They run in 2.5 and 3.2 ms per inference on a Raspberry Pi 4B, and the choice of INT8 kernels on the device changes between 0.3% and 9.4% of a model's predictions. The quantum kernel leaves nothing in the deployed model, and our evidence does not show an accuracy benefit over classical alternatives.

The results are exploratory. One question remains open: whether the quantum map beats classical maps of equal width. A confirmatory test would need a frozen protocol, the random-feature control we did not run, at least five seeds, and fresh data. On the deployment side, energy and further devices remain to be measured.

REFERENCES

[1] M. Yang and G. Thung, "Classification of trash for recyclability status," 2016, cS229 project report, Stanford University. TrashNet dataset.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.

[3] A. Howard *et al.*, "Searching for MobileNetV3," in *IEEE/CVF International Conference on Computer Vision*, 2019.

[4] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.

[5] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," 2018.

[6] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint*

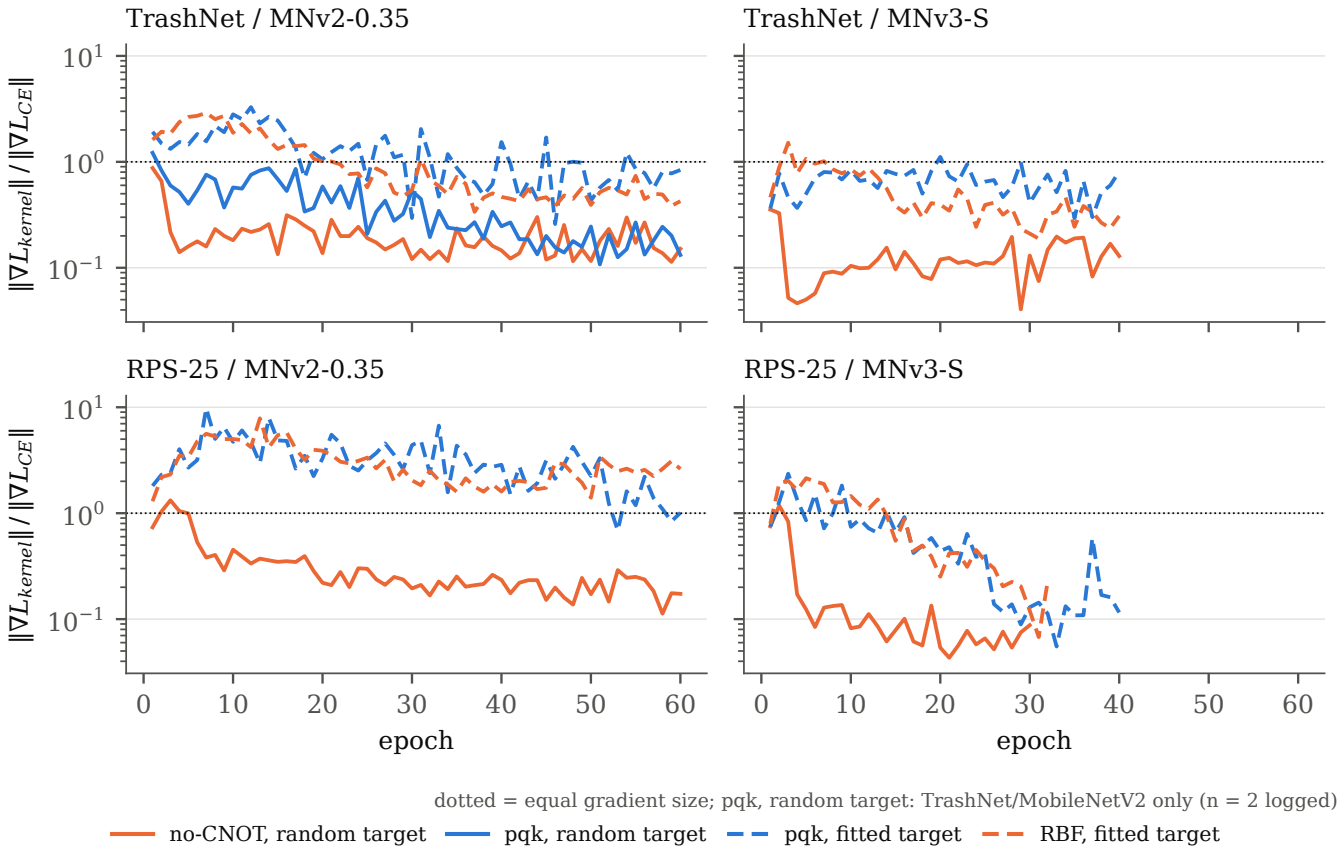


Fig. 9. Ratio of the gradient norm of the kernel term to that of the cross-entropy term, on the first batch of each epoch, averaged over seeds. Dashed lines use the fitted teacher projection. The solid blue line (`pqk` with a random target) was logged for TrashNet/MobileNetV2-0.35 only.

TABLE XIII

INT8 models on the Raspberry Pi 4B with the default XNNPACK delegate, four threads, batch size 1: median (p50) and 95th-percentile (p95) invocation time over 500 calls, mean throughput, and the speed-up from one to four threads. Latency, throughput and memory are averages over the methods and seeds of each pair (15–18 models); the thread speed-up is measured on one model per pair (`pqk`, seed 0). The spread of the p50 between methods is at most 0.11 ms within a pair. Sizes are in KiB.

Dataset	Student	Params	INT8 KiB	MMACs	p50 ms	p95 ms	FPS	1→4 thr.
TrashNet	MobileNetV2-0.35	403,814	608	18.9	2.53	2.71	395	1.84×
TrashNet	MobileNetV3-S	930,470	1181	18.2	3.18	3.35	314	1.35×
RPS-25	MobileNetV2-0.35	399,971	604	18.9	2.52	2.73	395	1.75×
RPS-25	MobileNetV3-S	928,739	1179	18.2	3.14	3.30	318	1.40×

arXiv:1503.02531, 2015.

[7] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for thin deep nets," in *International Conference on Learning Representations*, 2015.

[8] W. Park, D. Kim, Y. Lu, and M. Cho, "Relational knowledge distillation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 3967–3976.

[9] Y. Tian, D. Krishnan, and P. Isola, "Contrastive representation distillation," in *International Conference on Learning Representations*, 2020.

[10] A. Polino, R. Pascanu, and D. Alistarh, "Model compression via distillation and quantization," in *International Conference on Learning Representations*, 2018.

[11] J. Kim, Y. Bhalgat, J. Lee, C. Patel, and N. Kwak, "QKD: Quantization-aware knowledge distillation," 2019.

[12] C.-Y. Liu, K.-C. Chen, K. Murota, S. Y.-C. Chen, and E. Rinaldi, "Quantum relational knowledge distillation," 2025, preprint. URL recorded in the literature review: semantic-scholar.org/paper/b4cfa945df05c92671a2def8e0705c59c5c757a1. arXiv identifier not recorded [todo].

[13] M. Li, L. Fan, A. Cummings, X. Zhang, M. Pan, and Z. Han, "Hybrid quantum classical machine learning with knowledge distillation," in *IEEE International Conference*

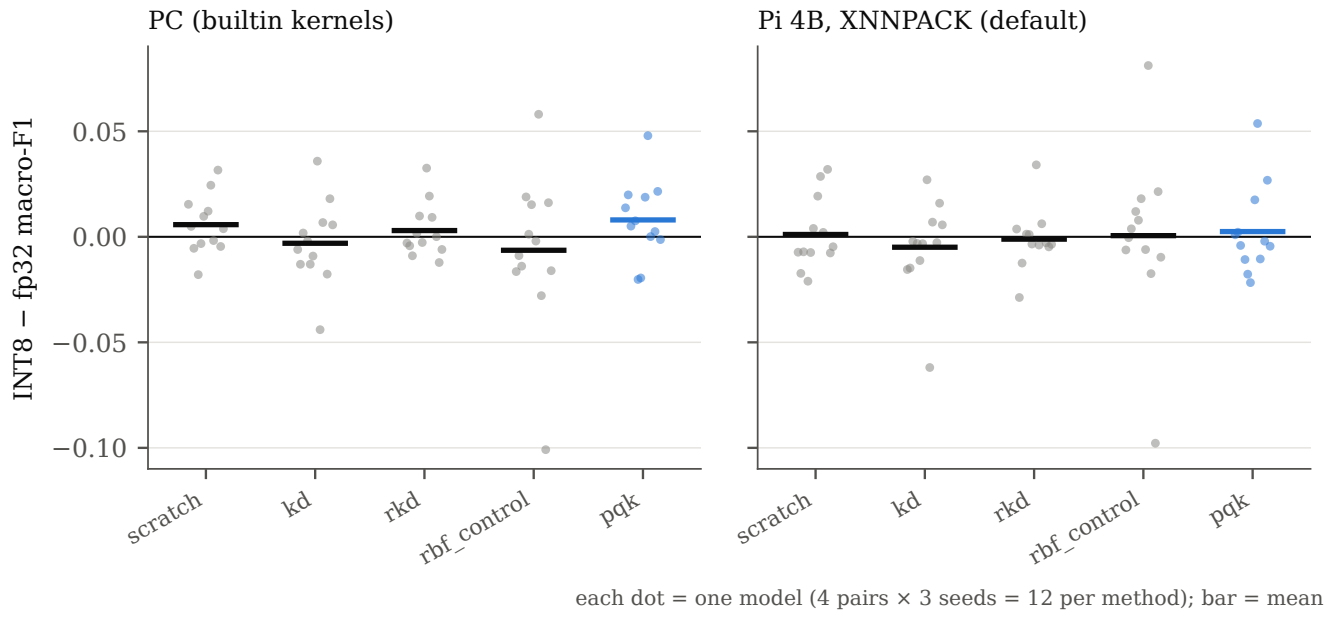


Fig. 10. INT8 minus fp32 test macro-F1 for every exported model, by method, on PC and with the Pi's default kernels. Each dot is one model; bars are means.

TABLE XIV
COMPUTE-HOURS OF THE RUNS BEHIND THE REPORTED RESULTS. RUNS SHARED THE GPU, SO THESE ARE COMPUTE-HOURS CONSUMED, NOT ELAPSED TIME, AND PER-EPOCH TIMES ARE NOT COMPARABLE BETWEEN METHODS. `PQK`-FAMILY HOURS INCLUDE THE CNN AND THE TEACHER FORWARD PASS, SO THEY UPPER-BOUND THE TIME SPENT SIMULATING THE CIRCUIT. TEACHER FINE-TUNING IS NOT INCLUDED.

Run family	Runs kept	Compute-hours
Classical (`scratch`, `kd`, `rkd`, `rbf_control`, controls)	98	49.8
`pqk` family (circuit simulator in the forward pass)	54	44.7
Discarded re-runs after pipeline fixes	–	23.0

on Communications, 2024.

[14] M. J. Hasan and M. R. C. Mahdy, "Bridging classical and quantum machine learning: Knowledge transfer from classical to quantum neural networks using knowledge distillation," arXiv:2311.13810, 2023.

[15] S. R. Naqvi, Z. Ding, F. Qi, M. M. Rahman, and L. Peng, "Structured supervision from quantum models: Distilling robustness into classical networks," in *AAAI Spring Symposium Series*, 2026.

[16] S. Thanasilp *et al.*, "Exponential concentration in quantum kernel methods," *Nature Communications*, vol. 15, 2024.

[17] L. Slattery *et al.*, "Numerical evidence against advantage with quantum fidelity kernels on classical data," *Physical Review A*, vol. 107, p. 062417, 2023.

[18] J. M. Kübler *et al.*, "The inductive bias of quantum kernels," in *Advances in Neural Information Processing Systems*, 2021.

[19] R. Flórez-Ablan *et al.*, "On the similarity of bandwidth-tuned quantum kernels and classical kernels," *Quantum Science and Technology*, 2025.

[20] J. Schnabel and M. Roth, "Quantum kernel methods under scrutiny: a benchmarking study," *Quantum Machine Intelligence*, 2025.

[21] H.-Y. Huang, M. Broughton, M. Mohseni, R. Babbush, S. Boixo, H. Neven, and J. R. McClean, "Power of data in quantum machine learning," *Nature Communications*, vol. 12, p. 2631, 2021.

[22] L. Moroney, "Rock, paper, scissors dataset," 2019, https://laurencemoroney.com/datasets.html.

[23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.

[24] Y. Liu *et al.*, "Knowledge distillation via instance relationship graph," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.

[25] B. He *et al.*, "Feature kernel distillation," 2022, venue not recorded in the literature review [todo].

[26] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, pp. 209–212, 2019.

[27] M. Schuld and N. Killoran, "Quantum machine learning in feature Hilbert spaces," *Physical Review Letters*, vol. 122, p. 040504, 2019.

[28] Y. Suzuki *et al.*, "Quantum fisher kernel for mitigating

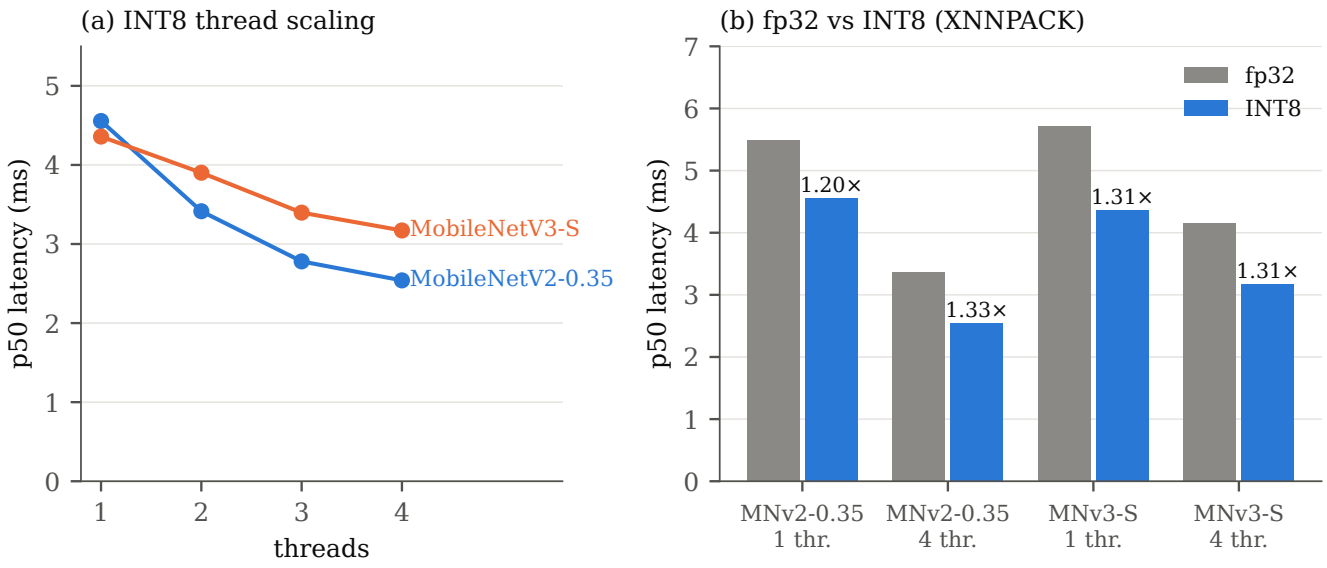


Fig. 11. (a) Median INT8 latency against the number of threads. (b) fp32 and INT8 latency with the same runtime, one and four threads, with the INT8 speed-up.

the vanishing similarity issue," *Quantum Science and Technology*, 2022.

[29] X. Wang *et al.*, "Towards understanding the power of quantum kernels in the NISQ era," *Quantum*, vol. 5, p. 531, 2021.

[30] S. Egginger *et al.*, "A hyperparameter study for quantum kernel methods," *Quantum Machine Intelligence*, 2023.

[31] E. Micklethwaite *et al.*, "Classification using quantum kernels in a radial basis function network," 2025.

[32] S. Kumar *et al.*, "Enabling edge intelligence: A quantum-inspired approach to deploying AI models on resource-constrained devices," in *ICEARS*, 2026, conference name abbreviated as in the DOI; full name not recorded in the review [todo].

[33] J. Violos, S. Papadopoulos, and Y. Kompatsiaris, "Towards optimal trade-offs in knowledge distillation for CNNs and vision transformers at the edge," arXiv:2407.12808, 2024.

[34] F. Paparounas *et al.*, "Evaluating knowledge distillation and compression techniques for edge class devices," 2024.

[35] Y. Matsubara *et al.*, "Head network distillation: Splitting distilled deep neural networks for resource-constrained edge computing systems," *IEEE Access*, 2020.

[36] V. Bergholm *et al.*, "PennyLane: Automatic differentiation of hybrid quantum-classical computations," 2018.

[37] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "RandAugment: Practical automated data augmentation with a reduced search space," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2020.

[38] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[39] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.